

CHP 6 TEXT PARSING

Text Parsing in NLP (Natural Language Processing)

Text parsing in NLP is the process of analyzing the structure of a sentence by breaking it down into its grammatical components, such as nouns, verbs, and phrases. Parsing helps to understand the syntactic structure of sentences, which is crucial for tasks like machine translation, information extraction, and question answering.

Key Points of Parsing:

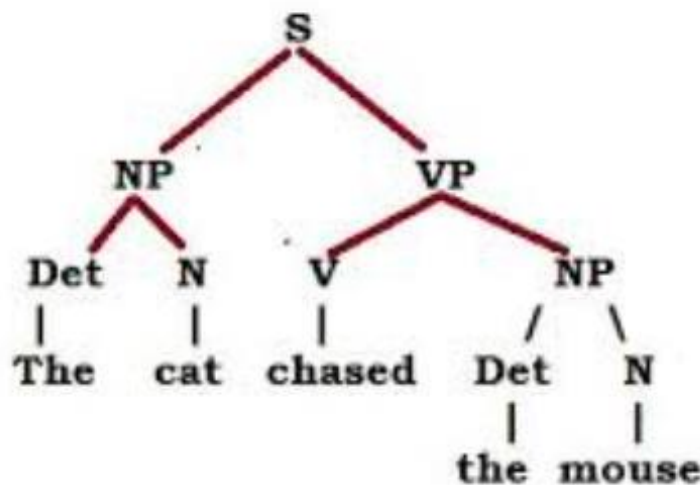
1. **Identify Grammatical Structure:** Parsing breaks down a sentence according to grammatical rules.
2. **Syntax Trees:** The result of parsing is often a syntax tree or parse tree, which shows the hierarchical structure of a sentence.
3. **Context-Free Grammars (CFG):** Parsing uses rules from grammars like CFG to determine the structure of the sentence.

Example of Text Parsing

Eg: "The cat chased the mouse."

When we parse this sentence, we want to break it down into its parts of speech and identify how the words relate to each other.

1. **Words in the Sentence:**
 - "The" (Determiner)
 - "cat" (Noun)
 - "chased" (Verb)
 - "the" (Determiner)
 - "mouse" (Noun)
2. **Parsing** The parsed result will be a **syntax tree** that looks like this:



The sentence is divided into a **Noun Phrase (NP)** and a **Verb Phrase (VP)**. Inside each phrase, words are categorized into their parts of speech. The **Verb Phrase** is further divided into the action ("chased") and another **Noun Phrase** ("the mouse").

Grammar Formalisms and Treebanks:

1. Grammar Formalisms

In Natural Language Processing (NLP), grammar formalisms provide the rules that govern how words and phrases combine to form sentences in a language. The formalism defines what constitutes a valid sentence in a language and is often expressed through grammar structures such as context-free grammars (CFGs) or other computational models.

Context-Free Grammar (CFG)

CFG is one of the most common types of grammar used in NLP. It consists of:

- **Terminals:** These are the actual words or tokens in the sentence (e.g., John, runs).
- **Non-terminals:** These represent syntactic categories such as noun phrases (NP) or verb phrases (VP).
- **Rules/Productions:** These specify how non-terminals can be expanded into terminals or other non-terminals. Each rule defines a structure for forming sentences.

Example of CFG Rules:

1. $S \rightarrow NP VP$

- A sentence (S) consists of a noun phrase (NP) followed by a verb phrase (VP).
2. $NP \rightarrow \text{Det } N$
 - A noun phrase consists of a determiner (Det) followed by a noun (N).
 3. $VP \rightarrow V \text{ NP}$
 - A verb phrase consists of a verb (V) followed by a noun phrase (NP).

Example Sentence and Grammar

Let's take the sentence: "**The cat eats fish.**"

Using the grammar formalism rules:

1. $S \rightarrow NP \text{ VP}$
2. $NP \rightarrow \text{Det } N$ (for "The cat")
3. $VP \rightarrow V \text{ NP}$ (for "eats fish")
4. $\text{Det} \rightarrow \text{the}, N \rightarrow \text{cat}, V \rightarrow \text{eats}, N \rightarrow \text{fish}$

This creates a tree structure that shows how the sentence is built from these components.

2. Treebanks

A **treebank** is a collection of sentences that are syntactically annotated. These annotations are often represented in the form of trees, where each node corresponds to a grammatical element (e.g., noun phrase, verb phrase).

Treebanks are essential for training NLP models that perform tasks like parsing, where the structure of sentences is identified. The tree structure helps in understanding how parts of the sentence relate to each other grammatically.

Example Treebank Structure for "The cat eats fish"

Let's visualize the tree using the rules described above:



- **S (Sentence)** is the root.
- **NP (Noun Phrase)** and **VP (Verb Phrase)** are expanded.
- **NP** further breaks down into **Det (Determiner)** and **N (Noun)**.
- **VP** expands into a **V (Verb)** and another **NP**.

Importance of Treebanks in NLP

- **Training Data:** Treebanks serve as training data for parsing algorithms, allowing the model to learn sentence structures.
- **Syntactic Parsing:** They help in tasks like syntactic parsing, where a machine automatically identifies the grammatical structure of sentences.
- **Dependency Analysis:** Treebanks also aid in dependency parsing, where the goal is to understand the dependencies between words (who does what to whom).

Commonly Used Treebanks

- **Penn Treebank:** One of the most famous treebanks, primarily for English.
- **Universal Dependencies (UD):** A cross-linguistic treebank for multiple languages.

Key Points to Remember:

- **Grammar Formalisms** provide rules for how sentences are structured in a language.
- **Treebanks** are annotated collections of sentences that provide syntactic trees to represent their structure.
- Both are fundamental for building NLP systems that understand and process language.

Parsing with Context-Free Grammars (CFG) :

Parsing with Context-Free Grammars (CFG) is a fundamental technique in NLP for analyzing the structure of sentences. By breaking down sentences into smaller components like noun phrases and verb phrases, CFG parsing helps machines understand how sentences are formed, making it a powerful tool for many NLP applications.

What is Context-Free Grammar (CFG)?

Context-Free Grammar (CFG) is a formalism used in NLP that defines how sentences in a language are structured. It consists of a set of rules, also called **productions**, which specify how sentences can be formed from smaller components.

Key Components of CFG:

- **Terminals:** These are the actual words in the sentence (e.g., "dog", "chases", "cat").
- **Non-terminals:** These are abstract symbols representing grammatical categories (e.g., noun phrase (NP), verb phrase (VP), sentence (S)).
- **Start Symbol:** The initial non-terminal from which parsing begins (typically "S" for a sentence).
- **Production Rules:** Rules that define how non-terminals can be expanded into terminals or other non-terminals.

$A \rightarrow B C$

This means non-terminal **A** can be expanded into non-terminals **B** and **C**.

CFG Rules Example

Let's define some CFG rules to describe simple English sentences:

$S \rightarrow NP VP$

A sentence (S) is formed by combining a noun phrase (NP) and a verb phrase (VP).

$NP \rightarrow Det N$

A noun phrase is made up of a determiner (Det) followed by a noun (N).

$VP \rightarrow V NP$

A verb phrase consists of a verb (V) followed by a noun phrase (NP).

$Det \rightarrow the \mid a$

The determiner can be "the" or "a".

$N \rightarrow dog \mid cat$

The noun can be "dog" or "cat".

$V \rightarrow chases \mid sees$

The verb can be "chases" or "sees".

Parsing a Sentence Using CFG

Let's parse the sentence:

"A dog chases the cat."

We will use the CFG rules to break this sentence into its grammatical components and construct a parse tree.

Step-by-Step Parsing:

Start with the sentence (S):

We begin by recognizing that the sentence is made up of a noun phrase (NP) followed by a verb phrase (VP), according to the rule:

$$S \rightarrow NP VP$$

Break NP and VP:

The noun phrase (NP) is "A dog", and the verb phrase (VP) is "chases the cat". The rule for NP says it consists of a determiner (Det) and a noun (N):

$$NP \rightarrow Det N$$

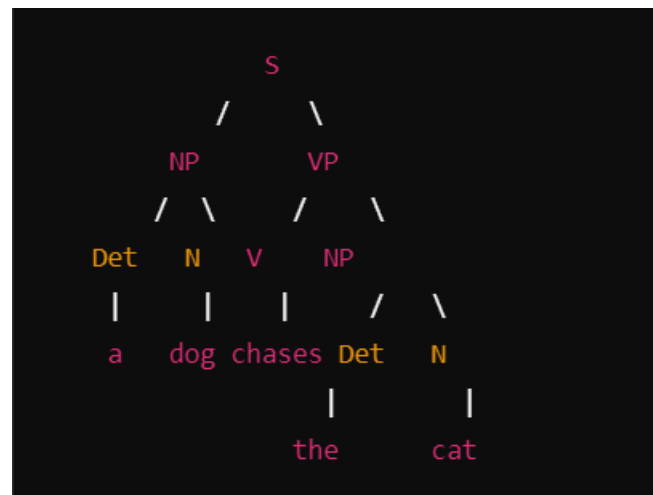
$$VP \rightarrow V NP$$

Further break NP into Det and N:

The determiner (**Det**) is "a", and the noun (**N**) is "dog". The verb (**V**) is "chases", and the second noun phrase (**NP**) "the cat" can be further broken into a determiner (**Det**) "the" and a noun (**N**) "cat".

4. Parse Tree for "A dog chases the cat"

The parse tree is a visual representation of how the sentence is constructed using the CFG rules. It shows how the sentence is broken down step by step.



5. Explanation of the Parse Tree:

S (Sentence) is the root of the tree.

The sentence is divided into two parts: **NP** (noun phrase) and **VP** (verb phrase).

The first **NP** ("A dog") consists of:

Det (Determiner): "a"

N (Noun): "dog"

The **VP** ("chases the cat") consists of:

V (Verb): "chases"

NP ("the cat"), which is further divided into:

Det (Determiner): "the"

N (Noun): "cat"

6. Why is Parsing Important in NLP?

Understanding Sentence Structure: Parsing helps machines understand how words in a sentence are related and arranged.

Syntactic Analysis: It provides a syntactic structure that can be used for tasks like machine translation, text summarization, and question answering.

Resolving Ambiguities: CFG-based parsers help resolve syntactic ambiguities by identifying the correct grammatical structure.

7. Applications of CFG Parsing in NLP:

Machine Translation: Understanding the structure of sentences is crucial for translating languages accurately.

Question Answering: Parsing helps in extracting meaning and relationships between words to answer questions accurately.

Sentiment Analysis: Parsing sentences helps in understanding the grammatical relationships between words, which can contribute to sentiment interpretation.

Features and Unification in NLP

In NLP, **Features and Unification** play an essential role in parsing and understanding language, especially when dealing with more complex grammatical structures where words need to agree in terms of certain properties like number (singular/plural), gender, tense, and case. These grammatical properties are called **features**.

Unification is the process used to ensure that these features are compatible with each other as we parse sentences. It is a fundamental concept in unification-based grammar formalisms, such as **Lexical Functional Grammar (LFG)** and **Head-Driven Phrase Structure Grammar (HPSG)**.

1. What are Features?

A **feature** is a grammatical property that a word or phrase possesses. Common features include:

- Number** (singular/plural): Describes whether a noun is singular (e.g., "cat") or plural (e.g., "cats").
- Person** (first, second, third): Describes the speaker's perspective (e.g., "I", "you", "he").
- Tense** (past/present/future): Describes when an action takes place.
- Gender** (masculine/feminine/neuter): Especially important in languages like Spanish or French, which have grammatical gender.
- Case** (nominative/accusative): Describes the function of a noun in a sentence, like subject or object.

2. What is Unification?

Unification is the process of combining or matching feature structures to ensure grammatical consistency. In parsing, each word or phrase has a feature structure (a set of features and their values), and when combining them (for example, a noun phrase and a verb phrase), the feature values must be compatible.

For instance:

- A singular subject must pair with a singular verb.
- A plural noun must pair with a plural determiner.

Unification is successful if the feature structures match or are compatible, and it fails if there is a conflict (e.g., trying to combine a singular noun with a plural verb).

3. Feature Structures

A **feature structure** is a way of representing the grammatical features of a word or phrase. It looks like a set of key-value pairs. For example:

For the word "**he**", the feature structure might look like this:

```
[ person: 3rd,  
  number: singular,  
  gender: masculine ]
```

For the word "**cats**", the feature structure might look like this:

```
[ number: plural,  
  gender: neuter ]
```

4. Example: Sentence Parsing with Features and Unification

Let's consider the following sentence for our example: "**The dog barks.**"

Step 1: Assign Feature Structures to Words

- "**The**" (determiner):

```
[ number: singular ]
```

"**dog**" (noun):

```
[ number: singular,  
  gender: neuter ]
```

"**barks**" (verb):

```
[ number: singular,  
  tense: present ]
```

Step 2: Unification During Parsing

As we parse the sentence "The dog barks", we need to ensure that the subject (**NP**) and the verb (**VP**) are compatible in terms of features.

1. **The** and **dog** must agree in number:

- **The**: [number: singular]
- **dog**: [number: singular, gender: neuter]

Since both have the feature **number: singular**, unification is successful, and we can combine them to form a noun phrase (NP):

NP: [number: singular, gender: neuter]

1. The noun phrase "**The dog**" must agree with the verb "**barks**" in number and tense:

- **The dog** (NP): [number: singular]
- **barks** (VP): [number: singular, tense: present]

Both the noun phrase and the verb phrase have **number: singular**, so unification succeeds again, and the sentence is valid grammatically.

Step 3: Final Feature Structure

After successful unification, the sentence has the following feature structure:

[number: singular,
tense: present,
gender: neuter]

5. What Happens if Unification Fails?

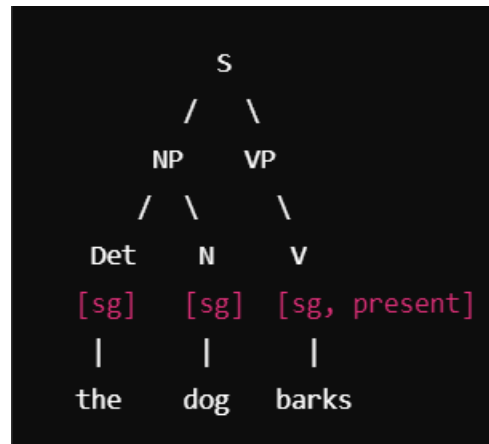
If we tried parsing the sentence "**The dogs barks**", unification would fail because:

- "**The dogs**" has the feature [number: plural].
- "**barks**" has the feature [number: singular].

Since these two feature structures conflict (singular vs. plural), unification would fail, and the sentence would be marked as ungrammatical.

6. Diagram Example

Let's visualize the sentence "**The dog barks.**" with its feature structures and unification process in the form of a simple parse tree:



In this tree:

The **Determiner (Det)** and **Noun (N)** in the noun phrase (**NP**) both have the feature **[number: singular]**.

The verb (**V**) also has the feature **[number: singular]** and **[tense: present]**.

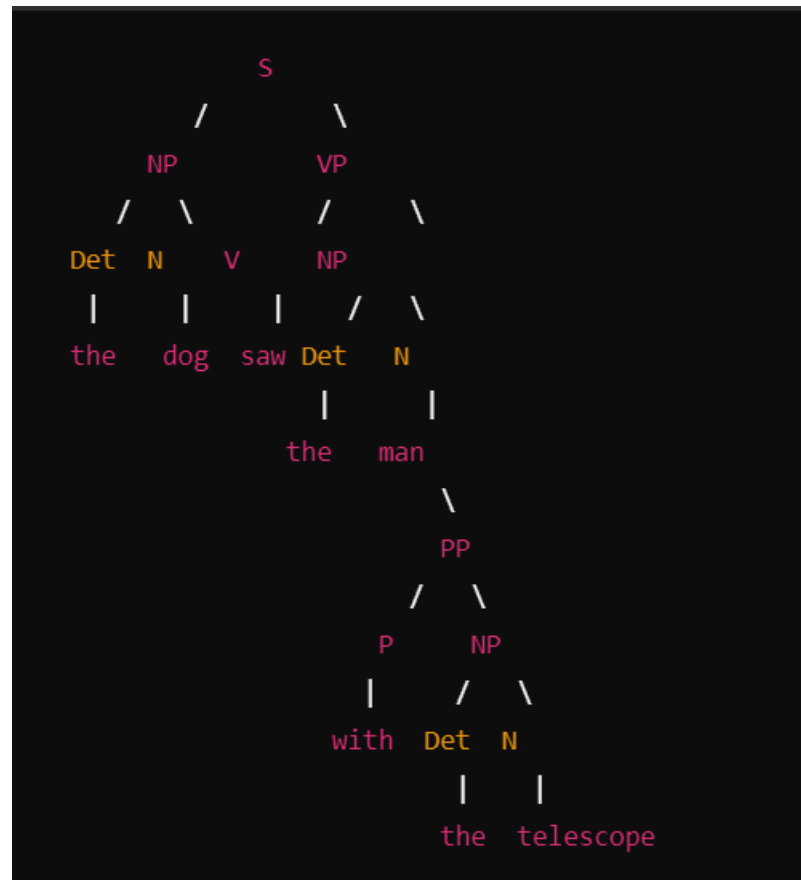
The NP and VP are unified, confirming that the sentence structure is valid.

7. Importance of Features and Unification in NLP

Grammatical Agreement: Features and unification help ensure that subjects agree with verbs, and determiners agree with nouns in number and gender, making sentences grammatically correct.

Handling Complex Languages: In languages with rich morphology (like German or Russian), unification helps in handling case, gender, and tense effectively.

Syntax and Semantics: Features help capture more than just syntax; they also reflect semantic information like tense (time) and person, which are important for understanding meaning.



8. Advantages of PCFGs

- **Disambiguation:** PCFGs help disambiguate sentences that have multiple possible interpretations by selecting the most likely parse based on probabilities.
- **Realistic Parsing:** PCFGs allow for more realistic parsing since they take into account how language is actually used (based on corpus data) rather than relying solely on rigid rules.
- **Probabilistic Models:** They provide a way to model uncertainty and variations in natural language, which is crucial for real-world applications like machine translation and speech recognition.

9. Challenges with PCFGs

- **Limited Context:** PCFGs only consider local context (immediate parent-child relationships in the tree), and they may not capture long-distance dependencies well (e.g., subject-verb agreement across a complex sentence).
- **Data Dependency:** The quality of the PCFG depends on the corpus used to estimate rule probabilities. A poorly trained PCFG will not perform well in parsing.